



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

TESI DI LAUREA MAGISTRALE IN
COMPUTER SCIENCE AND ENGINEERING
INGEGNERIA INFORMATICA

Binary Instrumentation for Tracking Edge Coverage in Fuzzing

Author:

Scandroglio Edoardo

Student ID:

XXXXXX

Advisor:

Prof. Name Surname

Academic Year:

2026-27

Dedicated to my family.

Abstract

ciao sono un abstract

Keywords: key, words, go, here

Abstract in lingua italiana

Ciao sono un sommario

Parole chiave: qui, vanno, le, parole, chiave

Contents

Abstract	i
Abstract in lingua italiana	iii
Contents	v
1 Background and Motivation	1
1.1 Fuzzing Background	1
1.2 Problem Statement	2
Bibliography	5
List of Figures	9
List of Tables	11
Acknowledgements	13

1 | Background and Motivation

1.1. Fuzzing Background

Fuzzing is nowadays a widely used approach for software testing and vulnerability assessment. Its main strength derives from the fact that it could rapidly produce various inputs to be fed into a target executable. In general a fuzzer can be divided into the **Input Generator**, that is responsible of providing inputs to the **Executor** that executes them. During the execution the **Defect Monitor** checks if the input executed ends in a crash of the target.

In particular the Input Generator must choose which input to provide to the executor between all the inputs in the *Input Space*. A subset of the *Input Space* is represented by the *Valid Input Space*. A good Input Generator must choose inputs that are also valid inputs, because only by doing this there is a probability of finding a vulnerability[5]. So the need of a precise input selection and mutation arises, in order to discover as many edge cases as possible.

In order to do so, the Input Generator must gain some information from the Executor. Depending on the amount of information that the Input Generator has, fuzzing can be classified into *White-box*, *Black-box* or *Grey-box*. In Black-box fuzzing the fuzzer does not have any knowledge about the execution.[add citation of black box fuzzer] In White-box fuzzing instead the source code must always be available and techniques of Symbolic and Concolic execution are used.[add citation on white box fuzzer]. In Grey-box fuzzing instead the inputs are chosen using, for example, **Code Coverage** information (i.e. number of lines of code visited during execution of the target). This approach is called *Coverage-Guided Grey-box fuzzing* and is based on prioritizing the inputs that discover new sections of code. The best approach to Code Coverage, in terms of overhead and effectiveness, is **Edge Coverage** which keeps track of which edges in the Control Flow Graph (CFG) are taken during execution. The coverage information is obtained through the instrumentation of the target application. Depending on the availability of the source code, the instrumentation could be added in different ways:

- The source code is available : In this case the instrumentation is added at *Compile*

Time

- The source code is not available : In this case there are two different techniques to instrument the application:
 - **Static Binary Instrumentation (SBI)** : The instrumentation is added before the target is executed but without having the source code, so relying on Binary Instrumentation techniques
 - **Dynamic Binary Instrumentation (DBI)** : The instrumentation is added during the program execution

1.2. Problem Statement

Coverage-Guided Grey-box fuzzing is one of the most effective techniques in automatic vulnerability disclosure on software, due to its capability of generating wide amount of inputs based on the feedback obtained through Code Coverage informations that guide the fuzzer towards unexplored regions of code and therefore find hidden vulnerabilities. However the effectiveness of this approach strongly depends on the availability of the source code. Infact both SBI and DBI have many limitations. The main limitation of DBI is represented by huge amount of overheads (higher than 600% overheads according to [4]) that makes the exploration of a substantial part of the valid input space almost impossible.[add citation to paper that state the high overheads] Instead SBI main difficulty is represented by the difficulty in modifying already compiled binaries. There are several problems regarding Binary Rewriting such as Indirect Control Transfers (e.g. `call rax`), shift in binary sections or instructions that cause the need of rewrite the instructions that reference these sections and so on.

For these reasons the preferable technique nowadays is *Compile time Edge Coverage instrumentation* with fuzzers such as AFL++ [1], LibAFL [2] o libFuzzer [3]. However in practice, for industrial secrecy (i.e. proprietary software) or due to precompiled libraries, the applications are often closed source, and so Compile Time Edge Coverage instrumentation become unusable.

In these scenario binary only fuzzing is the only viable approach. Since DBI has prohibitive overheads by definition, in this thesis I will address the problem of statically instrument a binary with Edge Coverage instructions in a way that the instrumented binary is compatible with AFL++ .

AFL++ has been choosen as fuzzer due to its popularity and because represent the current state of the art implementing all the state of the art techniques disclosed by researchers in last years [1].

The instrumentation must add as little overheads as possible with respect to compile time overheads and must preserve the correctness of the binary.

Bibliography

- [1] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse. AFL++ : Combining incremental steps of fuzzing research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association, Aug. 2020. URL <https://www.usenix.org/conference/woot20/presentation/fioraldi>.
- [2] A. Fioraldi, D. C. Maier, D. Zhang, and D. Balzarotti. Libafl: A framework to build modular and reusable fuzzers. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 1051–1065, 2022.
- [3] LLVM Project. libfuzzer – a library for coverage-guided fuzz testing. <https://llvm.org/docs/LibFuzzer.html>. Accessed: 2026-09-11.
- [4] S. Nagy, A. Nguyen-Tuong, J. D. Hiser, J. W. Davidson, and M. Hicks. Breaking through binaries: Compiler-quality instrumentation for better binary-only fuzzing. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 1683–1700. USENIX Association, Aug. 2021. ISBN 978-1-939133-24-3. URL <https://www.usenix.org/conference/usenixsecurity21/presentation/nagy>.
- [5] X. Zhu, S. Wen, S. Camtepe, and Y. Xiang. Fuzzing: A survey for roadmap. *ACM Computing Surveys*, 54(11s):1–36, Jan. 2022. ISSN 1557-7341. doi: 10.1145/3512345.

List of Figures

List of Tables

Acknowledgements

Here you may want to acknowledge someone.

